

GitHub link: <https://github.com/sudha-uni/cs516-hw1>

Technical description (1 page max):

For this assignment, I extended the moments web application with two machine-learning powered features: automatic alternative text generation for images and image search by recognized objects. The goal was to improve accessibility for visually impaired users and expand image search capabilities, while keeping the codebase modifications minimal and aligned with production-ready practices.

1. Alternative Text Generation:

When a user uploads an image, the system checks if a manual description (alt text) is provided. If not, the backend calls the Azure Computer Vision API (Image Captioning endpoint) to generate a caption. The returned caption is stored in the database as the image's description and inserted into the HTML `<img>` tag via the alt attribute. This ensures that all uploaded images have accessible alternative text. To achieve this, I integrated the Azure Computer Vision API (Image Captioning endpoint). Specifically, in the upload function of `moments/blueprints/main.py`, I added a conditional check: if the description field is empty, the image file is sent to the ML wrapper in `moments/utils/ml.py`, which calls the API, receives a caption, and stores it as the image description.

This value is persisted in the database and injected into the HTML templates, specifically in `templates/main/photos.html`, where the `<img>` tag now includes the alt attribute populated with either the user-provided or automatically generated description. This guarantees compliance with accessibility standards, since every displayed image now has alt text.

Relevant references:

- `moments/blueprints/main.py`, upload
- `moments/utils/ml.py` – captioning function
- `moments/templates/main/photo.html`

2. Image Search:

To allow keyword-based image retrieval, I extended the moments database schema with a new field `detected_tags` in `moments/models.py`. Whenever an image is uploaded, the object detection endpoint of the Azure Vision API is invoked. Detected objects (e.g., "cat", "mountain", "bicycle") are stored as a comma-separated string in `detected_tags`.

In the search route of `moments/blueprints/main.py`, I updated the SQLAlchemy query logic to search across both user-provided tags and automatically generated tags. This means that even if an uploader did not annotate the photo, users can still discover it by searching for terms matching the detected objects.

Relevant references:

- `moments/models.py` – added field `detected_tags`
- `moments/utils/ml.py` – object detection function
- `moments/blueprints/main.py`, `search`

3. Implementation Practices:

I made sure to follow best practices:

- No private credentials in the repo: API keys are stored in `.env` and loaded at runtime using `python-dotenv`.
- Documentation updates: The `README.md` contains instructions for installing dependencies, setting up environment variables, and running the system.
- Minimal invasive changes: Most of the ML logic is encapsulated in `utils/ml.py` to isolate external dependencies and make future updates easier.

User interface design approach (1 page max):

When introducing ML features, the challenge is balancing automation with user control. For both features, I carefully considered the five design interaction strategies (automate, prompt, organize, annotate, hybrid) in terms of forcefulness, frequency, value, and cost.

1. Alternative Text Generation:

Recommendation: Automate with optional override. Users rarely provide alt text, but it is critical for accessibility. Automatically generating alt text ensures every image is accessible by default, providing high value with low cost to the user. The process should remain invisible unless the user wants to intervene.

Forcefulness: Low, because users are not forced to provide text.

Frequency: High, because captions are needed for every image upload.

Value: High, improving accessibility and search.

Cost: Low, since generation is automatic.

Implementation vs. Recommendation: Currently, the feature is automated without an explicit override. If I had more time, I would add a prompt during upload: once the caption is generated, it appears in the description field where the user can accept or edit it. This hybrid approach preserves automation while respecting user agency.

2. Image Search:

Recommendation: Hybrid (organize + annotate). Detected tags should not only power the backend search but also be shown transparently to users. For example, images could display both user-provided and ML-generated tags, with AI-generated tags marked separately. This transparency builds trust and helps users understand why certain results appear.

Forcefulness: Moderate, since tags are generated whether the user wants them or not, but displayed transparently.

Frequency: Medium, depending on how often users search or browse.

Value: High, making images discoverable without manual tagging.

Cost: Low for users, since no action is required.

Implementation vs. Recommendation: In my implementation, the detected tags are stored and searchable but not displayed in the interface. With more time, I would expose them in the photo detail view with a subtle label (e.g., "AI-generated tags") so that users can validate and, if needed, correct them.

This design ensures both features maximize user benefit while minimizing friction.

Harms (1 page max):

Machine learning features introduce risks that go beyond technical failures. They can create social, ethical, and fairness-related harms.

1. Potential Harms:

- **Inaccuracy and Misrepresentation:** Automatically generated captions may be wrong or misleading, especially for nuanced images (e.g., confusing a wheelchair for a chair). This can harm visually impaired users who rely on alt text.
- **Bias and Offensive Output:** Vision models trained on internet data may reflect cultural stereotypes or offensive biases (e.g., mislabeling gender or race). If displayed without review, this can harm users and damage trust in the platform.
- **Privacy Risks:** Automatically tagging people or objects could inadvertently reveal sensitive information (e.g., identifying locations or personal possessions). Users may not want this metadata searchable.
- **Over-reliance on Automation:** If users assume AI-generated descriptions are always correct, they may skip adding more accurate human-written captions, reducing overall content quality.

2. Mitigation Strategies:

- **Transparency:** Add disclaimers near generated captions (e.g., “Alt text automatically generated and may contain errors”).
- **User Control:** Allow users to edit captions and delete AI-generated tags easily. This creates a correction loop.
- **Moderation Layer:** Filter generated captions and tags through a banned-words or sensitive-content checker before displaying them.
- **Feedback Loop:** Collect anonymous corrections from users and use them to improve model selection or fine-tune models in the future.
- **Privacy Controls:** Allow users to opt out of automatic tagging for sensitive images, providing a toggle in settings.

By explicitly surfacing limitations and giving users agency, harms can be reduced while preserving accessibility benefits.

Production challenges (1 page max):

Moving from a demo-scale project to production raises significant engineering and operational challenges such as:

1. Scalability:

Problem: At small scale, every image upload triggers an API call to Azure Vision. At millions of uploads per day, this could overwhelm the system and cause latency issues, since each call involves network overhead and model inference time.

Possible Solution: Use asynchronous processing with task queues such as Celery with Redis. The upload completes immediately, while captions/tags are generated in the background. This decouples user experience from ML latency.

2. Operating Costs:

Problem: Cloud providers like Azure charge per API request. At scale, generating captions and tags for millions of images could cost thousands of dollars monthly.

Possible Solution: Adopt a hybrid inference model where you use a lightweight local model (e.g., Hugging Face BLIP for captions, CLIP for embeddings) for bulk processing, and fall back to paid APIs only for images where higher accuracy is needed while implementing caching to avoid duplicate requests for identical images.

3. Reliability and Monitoring:

Problem: Dependence on third-party APIs introduces risks (rate limits, outages). Without monitoring, failures may silently degrade user experience.

Possible Solution: Add telemetry and error handling. If an API call fails, store a placeholder alt text (“Image could not be captioned”) and log the failure. Use monitoring dashboards (e.g., Prometheus + Grafana) to track request success rates, costs, and latency.

4. Data Management:

Problem: Storing millions of detected tags increases database size and search complexity. Queries may become slow.

Possible Solution: Use inverted indices or specialized search tools (e.g., Elasticsearch) to store tags and support fast keyword queries at scale.

By addressing these production challenges—scalability, cost, reliability, and data management—the system can grow beyond a demo project into a realistic large-scale platform.

